

AIA Unit 4 Notes

1. Heuristic Search

1.1 Heuristic Functions

A heuristic function $h(n)$ is an estimate of the cost from node n to the goal state. It provides problem-specific knowledge that guides the search towards the goal more efficiently — without guaranteeing perfection.

Properties of a good heuristic:

- **Admissibility:** $h(n)$ never over-estimates the true cost to the goal: $h(n) \leq h^*(n)$. Guarantees optimality in A*.
- **Consistency (Monotonicity):** $h(n) \leq \text{cost}(n, n') + h(n')$ for every successor n' . Ensures A* never re-expands a node.
- **Informedness:** A more informed heuristic (closer to true cost) leads to less exploration and faster search.

Common Heuristic Examples:

Map Routing:	$h(n)$ = straight-line distance to goal	(admissible — can't travel faster)
8-Puzzle:	$h(n)$ = number of misplaced tiles	(admissible — each tile needs ≥ 1 move)
8-Puzzle:	$h(n)$ = sum of Manhattan distances	(admissible and more informed)

► **Heuristic Search & Functions:** <https://www.youtube.com/watch?v=xaVTpt58bFI>

1.2 Hill Climbing

Hill Climbing is a local search algorithm that continuously moves to the neighbouring state with the highest value (or lowest cost). It never backtracks — it only moves uphill, like climbing a hill in fog where you can only see your immediate surroundings.

Algorithm:

1. Start from an initial state.
2. Evaluate all neighbouring states using the objective/heuristic function.
3. Move to the best neighbour (highest value / lowest cost).
4. Repeat until no neighbour is better than the current state → local optimum reached.

Problems with Basic Hill Climbing:

Problem	Description	Example
Local Maximum	A state better than all neighbours but not the global best. Algorithm stops here.	Stuck on a small hill instead of the mountain peak
Plateau	A flat region where all neighbours have equal value. No direction to move.	All roads look equally good — paralysed
Ridge	A sequence of local maxima where diagonal movement is needed but only straight moves allowed.	Staircase-like terrain where direct climbing fails

Variants that overcome these problems:

- **Stochastic Hill Climbing:** Randomly selects among uphill moves — adds randomness to escape some plateaus.
- **First-Choice Hill Climbing:** Randomly generates successors until one better than current is found — good when many successors exist.
- **Random-Restart Hill Climbing:** Restarts from randomly chosen initial states multiple times — keeps best result found. Overcomes local maxima.
- **Simulated Annealing:** Occasionally accepts worse moves (with decreasing probability over time) to escape local optima. Inspired by metallurgy.

► **Hill Climbing Algorithm:** <https://www.youtube.com/watch?v=z2y1sMrHKDw>

1.3 Variable Neighborhood Descent (VND) (not in ppt)

Variable Neighborhood Descent is a local search improvement where the search systematically changes the neighbourhood structure when a local optimum is reached — instead of giving up or restarting randomly.

Key Idea:

A solution that is locally optimal for one neighbourhood structure may not be locally optimal for a different neighbourhood. By switching neighbourhoods, VND can escape local optima.

VND Algorithm:

1. Define a set of neighbourhood structures $N_1, N_2, \dots, N_k$ (ordered by size or complexity).
 2. Start with neighbourhood N_1 . Find the best neighbour in N_1 .
 3. If improvement found → move to that neighbour, reset to N_1 .
 4. If no improvement in N_i → switch to N_{i+1} (next neighbourhood).
 5. Repeat until no improvement found in any neighbourhood → local optimum across all structures.
- **Advantage over simple Hill Climbing:** Systematically diversifies the search by using multiple neighbourhoods — far more likely to escape local optima.
 - **Applications:** Combinatorial optimisation, vehicle routing, scheduling, graph colouring.

1.4 Beam Search

Beam Search is a heuristic search algorithm that navigates a graph by systematically expanding the most promising nodes within a constrained set. It combines elements of BFS but only keeps the best W nodes (the 'beam') at each level.

Key Parameters:

- **Beam Width (W):** The number of nodes kept at each level. Larger W = better exploration but more memory/time.
- **Branching Factor (B):** At each level, $W \times B$ nodes are evaluated but only W are selected for expansion.

How Beam Search Works:

1. Initialise: Start with the root node and generate its successors.
2. Evaluate: Apply the heuristic function to all generated successors.
3. Select: Keep only the top W nodes by heuristic value — discard the rest.
4. Iterate: Expand selected nodes, generate their successors, evaluate and select again.
5. Terminate: Stop when goal is found, or no more nodes remain to expand.

Property	Detail
Complete?	Not guaranteed — pruning may eliminate the optimal path
Optimal?	Not guaranteed — especially with narrow beam width
Memory	$O(W \times B)$ — bounded and predictable
Time	Better than exhaustive BFS — proportional to W and depth
$W = 1$	Equivalent to Greedy Best-First (Hill Climbing)
$W = \infty$	Equivalent to full BFS — explores everything

Advantages:

- **Efficiency:** Navigates large search spaces far more efficiently than exhaustive search.
- **Flexibility:** Adjusting beam width trades off solution quality against speed and memory.
- **Scalability:** Does not store all nodes — suitable for resource-constrained environments.

Limitations:

- **Suboptimality:** Narrow beam width may prune the optimal path — no guarantee of best solution.
- **Heuristic dependency:** Poor heuristics lead to poor node selection and bad results.

Applications:

- **NLP:** Machine translation and speech recognition — finding best word sequence.
- **Robotics:** Efficient pathfinding in large environment maps.
- **Game AI:** Strategic games where exploring every move is computationally infeasible.

1.5 Tabu Search (not in ppt)

Tabu Search is a metaheuristic local search method that uses a memory structure called the tabu list to avoid revisiting recently explored solutions — allowing the search to escape local optima by accepting temporarily worse moves.

Core Components:

- **Current Solution:** The solution being evaluated at each step.
- **Neighbourhood:** The set of solutions reachable from the current solution by a single move.
- **Tabu List:** A short-term memory of recently visited solutions or moves. Moves on this list are forbidden (tabu) for a number of iterations.
- **Aspiration Criterion:** Exception rule: a tabu move can be accepted if it leads to a solution better than the best found so far.
- **Tenure:** The number of iterations a move remains on the tabu list before expiry.

Tabu Search Algorithm:

1. Start with an initial solution s . Set $s_best = s$.
2. Generate neighbourhood $N(s)$ — all solutions reachable in one move.
3. Select the best non-tabu solution s' from $N(s)$. (Or accept tabu move if aspiration criterion met.)
4. Update tabu list — add the move that led to s' .
5. If s' is better than s_best , update $s_best = s'$.
6. Set $s = s'$. Repeat from Step 2 until stopping condition met.

Tabu Search Key Insight

Unlike Hill Climbing which stops at any local optimum, Tabu Search can move to a worse solution to escape it. The tabu list prevents cycling back — forcing exploration of new areas of the solution space.

- **Applications:** Job scheduling, vehicle routing, network design, VLSI design, travelling salesman problem.

1.6 Optimal Search — A* Algorithm

A* (A-Star) is the most widely used informed search algorithm. It finds the optimal (shortest / lowest-cost) path from start to goal by combining the actual cost to reach a node with a heuristic estimate of the remaining cost.

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start to n

$h(n)$ = heuristic estimate from n to goal

$f(n)$ = estimated total cost of path through n

OPEN and CLOSED Lists:

- **OPEN List:** The frontier — nodes discovered but not yet fully evaluated. A* always expands the node with the lowest $f(n)$ from OPEN.
- **CLOSED List:** Nodes already expanded. A* does not re-expand nodes in CLOSED (with a consistent heuristic).

A* Algorithm Steps:

1. Add start node to OPEN list with $f(\text{start}) = h(\text{start})$.
2. If OPEN is empty $\rightarrow$ no solution exists. Stop.
3. Remove node n with lowest $f(n)$ from OPEN. Add n to CLOSED.
4. If n is the goal $\rightarrow$ solution found! Reconstruct path by tracing parent pointers.
5. Expand n : for each successor s of n :
 - a. If s is in CLOSED $\rightarrow$ skip.
 - b. Compute $f(s) = g(s) + h(s)$. If s not in OPEN or new $f(s) <$ old $f(s)$: add/update s in OPEN.
6. Go to Step 2.

Pruning the OPEN and CLOSED Lists:

- **OPEN list pruning:** When a node is added to OPEN, if a better path to it is already known, the worse entry is discarded.
- **CLOSED list pruning:** When a node is in CLOSED with a consistent heuristic, it never needs to be re-expanded — pruning is safe.
- **Why pruning matters:** Without pruning, the lists grow unboundedly. Pruning keeps memory manageable and speeds up the search.

Property	Detail
Complete?	Yes — always finds a solution if one exists
Optimal?	Yes — if $h(n)$ is admissible (never over-estimates)
Time	Exponential in worst case — depends on heuristic quality
Space	$O(b^d)$ — stores all generated nodes
Key advantage	Guaranteed optimal with admissible $h(n)$ — best of both BFS and greedy

► **A* Algorithm:** <https://www.youtube.com/watch?v=95s0b0HBe3w>

1.7 Iterative Deepening A* (IDA*) (not in ppt)

IDA* combines the memory efficiency of Iterative Deepening DFS with the heuristic guidance of A*. Instead of maintaining an OPEN list (which can be huge), IDA* performs a depth-limited DFS, cutting off branches where $f(n) >$ threshold.

IDA* Algorithm:

1. Set threshold = $h(\text{start})$.
2. Perform DFS from start, expanding nodes only where $f(n) \leq$ threshold.

3. If goal found → return solution.
4. If no solution found within threshold → set threshold = minimum $f(n)$ value that exceeded the cut-off.
5. Repeat from Step 2 with new (higher) threshold.

Feature	A*	IDA*
Memory	$O(b^d)$ — stores all nodes	$O(bd)$ — only current path
Optimal?	Yes (admissible h)	Yes (admissible h)
Complete?	Yes	Yes
Re-expansion	No (CLOSED list)	Yes — may re-expand nodes each iteration
Best when	Memory is available	Memory is severely limited

Key Insight — IDA*

IDA* trades time for memory. It may re-visit nodes across iterations, but never stores the entire frontier — making it practical for problems where A* runs out of memory.

1.8 Recursive Best First Search (RBFS) (not in ppt)

Recursive Best First Search is a memory-efficient version of A* that uses recursion to simulate best-first search while using only linear memory $O(bd)$. It remembers the best alternative f -value at each level to know when to backtrack.

How RBFS works:

1. Expand the node with the best (lowest) f -value recursively.
 2. Track the best alternative path f -value at each recursive call.
 3. If the current node's f -value exceeds the alternative limit → backtrack, passing the best f -value seen back up.
 4. When backtracking, update the f -value of the abandoned node to the best f seen in its subtree.
 5. This allows RBFS to 'remember' how promising an abandoned branch was — and return to it later if better paths fail.
- **Memory:** $O(bd)$ — linear, like IDA*
 - **Optimal?:** Yes — with admissible heuristic
 - **Advantage over IDA*:** Avoids redundant re-expansion of nodes by storing the best-known f -value for each abandoned branch
 - **Disadvantage:** Frequent backtracking can make it slow in practice

2. Problem Decomposition

2.1 What is Problem Decomposition?

Problem decomposition is an AI problem-solving approach that breaks a complex problem into smaller, manageable sub-problems, solves each independently, and combines the results into a final solution.

1. Break / Divide the problem into various sub-problems.
 2. Find the solution for each sub-problem independently.
 3. Combine sub-solutions to arrive at the final solution.
- **Why useful:** Complex problems are often too large to solve directly. Decomposition makes them tractable by exploiting structure.
 - **Key insight:** If sub-problems are independent (don't share resources or constraints), solving them separately and combining is both correct and efficient.

2.2 Goal Trees (AND-OR Trees)

Goal trees represent problem decomposition visually. For each problem, there is a goal node. The tree is built by asking: 'Do we need to solve ALL of these sub-problems, or just ONE?'

Two types of nodes:

- **OR Node:** The problem can be solved by achieving ANY ONE of the alternatives (sub-goals). Represents choice.
- **AND Node:** The problem requires ALL of the sub-goals to be solved simultaneously. Represents decomposition.

AND-OR Tree Example – Getting to Work:

```
Goal: Get to Work  [OR node]
|
├── Drive Car  [AND node – needs both]
│   ├── Car has fuel
│   └── Roads are clear
├── Take Bus    [OR node – any one bus route]
│   ├── Route 10
│   └── Route 22
└── Cycle
```

The top goal is solved if ANY branch (Drive/Bus/Cycle) succeeds.
'Drive Car' is solved only if BOTH sub-goals are met.

Searching AND-OR Trees:

- **AO* Algorithm:** The standard algorithm for searching AND-OR graphs. Finds the optimal solution tree — considering both the cost of AND branches (sum of children) and OR branches (min of children).
- **Solved node:** A node is solved if: OR node — at least one child is solved; AND node — all children are solved.

► **Problem Decomposition & Goal Trees:** <https://www.youtube.com/watch?v=JL9g1BBSunA>

2.3 Rule-Based Systems

A Rule-Based System (also called a Production System) represents knowledge as a collection of IF-THEN rules stored in a knowledge base, and applies them to a working memory of known facts using an inference engine.

Three core components:

- **Knowledge Base (Rules):** IF <condition> THEN <action/conclusion>. The stored expert knowledge.
- **Working Memory (Fact Base):** The set of currently known facts about the world — updated as rules fire.
- **Inference Engine:** The control mechanism that matches rule conditions against working memory and decides which rules to fire.

Example Rules:

```
Rule 1:  IF temperature > 38.5°C    THEN diagnose(fever)
Rule 2:  IF fever AND cough          THEN recommend(flu_test)
Rule 3:  IF flu_test = positive      THEN prescribe(antiviral)
```

Working Memory initially: { temperature = 39.2, cough = true }

```
→ Rule 1 fires → adds fever to WM
→ Rule 2 fires → adds recommend(flu_test) to WM
→ Rule 3 fires → adds prescribe(antiviral) to WM
```

Inference Strategies:

- **Forward Chaining (Data-Driven):** Start from known facts, apply matching rules, add new conclusions — repeat until goal reached or no rules apply. Best for monitoring/alerting systems.
- **Backward Chaining (Goal-Driven):** Start from the goal, find rules that conclude the goal, establish their conditions as sub-goals — repeat until sub-goals are satisfied by known facts. Best for diagnosis and query answering.

Conflict Resolution (when multiple rules match):

- **Specificity:** Fire the most specific rule (most conditions in LHS).
- **Recency:** Prefer rules that match the most recently added facts.
- **Priority / Salience:** Each rule has a priority number — fire highest.
- **Refractoriness:** A rule cannot fire on the same data twice — prevents infinite loops.

2.4 Rule-Based Expert Systems

A Rule-Based Expert System is a type of AI system that uses a large rule base encoding the knowledge of human domain experts to solve complex problems in a specific domain — providing advice, diagnosis, or decisions at expert level.

Architecture of an Expert System:

Component	Description	Example
Knowledge Base	Collection of IF-THEN rules encoding expert knowledge	Medical diagnostic rules from doctors
Working Memory	Current facts about the specific problem being solved	Patient symptoms and test results
Inference Engine	Applies rules to facts to derive conclusions	Matches symptom rules, chains to diagnosis
Explanation Facility	Explains how a conclusion was reached — shows the chain of rules fired	'Flu diagnosed because: fever + cough + positive test'
Knowledge Acquisition Interface	Tool for experts to add/modify rules without programming	GUI for doctors to add new diagnostic rules
User Interface	Allows non-expert users to interact with the system	Questions presented to patient in natural language

Classic Expert System Examples:

- **MYCIN**: Medical diagnosis of bacterial infections — prescribed antibiotics at physician level.
- **DENDRAL**: Identified chemical structures from mass spectrometry data.
- **XCON (R1)**: Configured DEC VAX computer systems — saved millions annually.

Advantages and Limitations:

Advantages	Limitations
Transparent — rules are human-readable	Knowledge acquisition bottleneck — experts are hard to interview
Explainable — shows reasoning chain	Cannot learn — rules must be manually updated
Consistent — applies rules the same way every time	Brittle — fails gracefully on situations not in rule base
Accessible — encodes rare expertise	Rule explosion — large domains need thousands of rules

3. Planning

3.1 What is Planning in AI?

Planning is the process by which an AI agent determines a sequence of actions to achieve a desired goal from an initial state. It is considered the logical side of acting — deciding what tasks need to be performed and in what order.

- Everything humans do is with a definite goal in mind, and all actions are oriented toward achieving that goal. AI planning mirrors this.
- A plan is a sequence of actions where each action has preconditions that must be satisfied before it can be executed, and effects (positive and negative) that result from it.
- Planning requires: a domain description, a task specification, and a goal description.

Planning vs Search

Search explores a space of world states to find a path to the goal. Planning works at a higher level of abstraction — using symbolic descriptions of actions (with preconditions and effects) to reason about what needs to be done, not just find any path.

3.2 STRIPS

STRIPS (Stanford Research Institute Problem Solver) was created by Richard Fikes and Nils Nilsson in the late 1960s. It is one of the most significant and foundational methods for automated AI planning, and served as the basis for most modern AI planning systems.

The formal STRIPS language was created to govern robot movements in a manipulable environment. Its main focus is plan generation — the series of operations that take a system from its starting state to a desired target state.

Key Components of STRIPS:

- **States:** Described by a set of logical facts (positive literals) that are currently true.
- **Goals:** A set of conditions describing the desired result — what must be true at the end.

Every STRIPS Action has three parts:

- **Preconditions:** Conditions that must ALL be true before the action can be executed.
- **Add Effects (Add List):** Facts that become TRUE after the action is executed.
- **Delete Effects (Delete List):** Facts that become FALSE after the action is executed.

STRIPS Action Format:

```
Action: <name>
Preconditions: <list of facts that must be true>
Add List:      <facts that become true after>
Delete List:   <facts that become false after>
```

```
Example — Picking up a block:
Action: PICK-UP(x)
Preconditions: ON-TABLE(x), CLEAR(x), ARM-EMPTY
Add List:      HOLDING(x)
Delete List:   ON-TABLE(x), CLEAR(x), ARM-EMPTY
```

Heuristics and Symbols in STRIPS:

- **Symbols:** Represent knowledge so AI systems can process incoming data and establish relationships.
- **Heuristics:** Techniques that help solve problems in a feasible amount of time — may find approximate or optimal solutions.
- **Significance:** STRIPS provides a structured means of arriving at decisions — sequential action planning with a logical inference engine.

Planning with STRIPS — Steps:

1. Define the Initial State: Where the system currently is.
2. Set the Goal State: What the system should achieve.
3. Develop Actions: Each defined by preconditions and add/delete effects.
4. Search for Solutions: Using strategies like backward chaining from goal to initial state.

► **Linear Planning:** <https://www.youtube.com/watch?v=WIG0H0u8aCg>

► **Non-Linear Planning:** https://www.youtube.com/watch?v=kHwjIOZe_jY

3.3 Forward and Backward State Space Planning

Forward State Space Planning (FSSP) — Progression

FSSP starts from the initial state and applies actions forward to generate new states, continuing until the goal state is reached. It behaves like forward state-space search.

```
FSSP Process:
Start: Initial State S
      ↓ [apply action A1]
State S1
      ↓ [apply action A2]
State S2 ... until goal conditions are met
```

- **Also called:** Progression planning
- **Best when:** Few applicable actions at each state, well-defined initial state
- **Challenge:** Can generate many irrelevant states before reaching the goal

Backward State Space Planning (BSSP) — Regression

BSSP starts from the goal state and works backward — finding actions whose effects contain the goal conditions, then establishing those actions' preconditions as new sub-goals. Continues until the initial state is reached.

BSSP Process:

Start: Goal Conditions G

↑ [which action achieves G? → Action An]

Sub-goal: preconditions of An

↑ [which action achieves these? → Action An-1]

... until sub-goals match initial state

- **Also called:** Regression planning
- **Best when:** Goal is specific, many objects in domain — backward search focuses only on relevant facts

Feature	Forward (FSSP)	Backward (BSSP)
Direction	Initial state → Goal	Goal → Initial state
Explores	All reachable states	Only states relevant to the goal
Relevance	May explore irrelevant states	More focused — only goal-relevant actions
Best for	Simple domains, few actions	Complex domains, specific goals

3.4 Goal Stack Planning (not in ppt)

Goal Stack Planning (also known as STRIPS-style planning) uses a stack to manage both goals and actions. It interleaves goal selection and action execution to achieve the overall goal.

How Goal Stack Planning works:

1. Push the top-level goal onto the stack.
2. Pop the top item from the stack.
3. If it is a fact already true in the current state → satisfied, continue.
4. If it is a goal → find an action that achieves it. Push the action, then its preconditions as new goals.
5. If it is an action → execute it, update the current state.
6. Repeat until the stack is empty → plan complete.

Goal Stack Example (simplified):

Stack: [ON(A,B)] Current state: A,B,C on table

→ Find action: STACK(A,B). Push: [STACK(A,B), HOLDING(A), CLEAR(B)]

→ Achieve HOLDING(A): push PICK-UP(A) and its preconditions

→ Execute PICK-UP(A) → state updates

→ Execute STACK(A,B) → ON(A,B) is now true → goal satisfied

- **Problem:** The Sussman Anomaly — goal stack planning with interacting goals can undo progress. E.g., achieving ON(A,B) then ON(B,C) may require undoing ON(A,B).

- **Solution:** Non-linear / partial-order planning handles interacting sub-goals correctly.

3.5 Plan Space Planning (Partial-Order Planning) (not in ppt)

Plan Space Planning operates in the space of partial plans rather than world states. Instead of fixing a total order of actions upfront, it builds a plan by adding only the ordering constraints that are strictly necessary — leaving unrelated actions unordered (the Least Commitment principle).

Key Elements of a Partial-Order Plan:

- **Steps:** The set of plan actions, plus dummy START and FINISH steps.
- **Ordering Constraints:** $S_i < S_j$ means step S_i must come before S_j .
- **Causal Links:** $S_i \rightarrow P S_j$ means step S_i achieves precondition P for step S_j .
- **Open Preconditions:** Preconditions not yet causally supported — the planner must find actions to achieve these.

Threats and Resolution:

- **Threat:** A step S_k that might delete a condition P needed by a causal link $S_i \rightarrow P S_j$.
- **Promotion:** Put S_k after S_j ($S_k < S_j$ ordering) — so it runs after P is consumed.
- **Demotion:** Put S_k before S_i ($S_k < S_i$ ordering) — so it runs before P is needed.

Why Plan Space Planning?

Goal Stack (total-order) planning fixes a complete sequence even when order doesn't matter — unnecessarily rigid. POP commits only to necessary orderings, supporting parallel execution and correctly handling interacting sub-goals (solving the Sussman Anomaly).

3.6 A Unified Framework for Planning (not in ppt)

A Unified Framework for Planning recognises that forward search, backward search, and plan-space search are all instances of searching through a space — just different spaces with different representations.

Planning Approach	Search Space	Key Idea	Handles Interaction?
Forward State Space (FSSP)	World states	Apply actions forward from initial state	No — total order
Backward State Space (BSSP)	World states	Regress from goal, establish preconditions	No — total order
Goal Stack Planning	Stack of goals/actions	Push goals, pop and achieve them	Limited — Sussman anomaly
Plan Space (POP)	Partial plans	Add steps and constraints as needed	Yes — least commitment

Planning Approach	Search Space	Key Idea	Handles Interaction?
Hierarchical (HTN)	Abstract task networks	Decompose high-level tasks into primitives	Yes — structured

In the unified view, all planning is search — the choice of representation (state space vs plan space) and direction (forward vs backward) determines efficiency and expressiveness.

4. Constraint Satisfaction

4.1 What is Constraint Satisfaction?

A Constraint Satisfaction Problem (CSP) consists of a set of variables, each with a domain of possible values, and a set of constraints that specify which combinations of values are allowed. The goal is to assign a value to every variable such that all constraints are satisfied.

Formal Definition:

- **Variables:** $X = \{X_1, X_2, \dots, X_n\}$ — the unknowns to be assigned.
- **Domains:** $D = \{D_1, D_2, \dots, D_n\}$ — the set of possible values for each variable.
- **Constraints:** $C = \{C_1, C_2, \dots, C_m\}$ — restrictions on allowed combinations of variable values.
- **Solution:** An assignment of values to all variables that satisfies every constraint.

Classic CSP Example — Map Colouring:

Variables: WA, NT, SA, Q, NSW, V (Australian states)

Domains: {Red, Green, Blue}

Constraints: Adjacent states must have different colours

e.g., $WA \neq NT$, $WA \neq SA$, $NT \neq SA$, $NT \neq Q$, $SA \neq Q$...etc

Solution: WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red, T=Red

Other CSP Examples:

- **N-Queens:** Place N queens on N×N board — no two queens attack each other.
- **Sudoku:** Fill 9×9 grid so each row, column, and 3×3 box contains digits 1–9 exactly once.
- **Scheduling:** Assign time slots to tasks respecting resource and precedence constraints.
- **Graph Colouring:** Colour nodes of a graph so no two adjacent nodes share a colour.

4.2 N-Queens Problem

The N-Queens problem is to place N chess queens on an N×N chessboard such that no two queens attack each other — no two queens share the same row, column, or diagonal.

N-Queens as a CSP:

Variables: $Q_1, Q_2, \dots, Q_N$ (one queen per column)

Domains: $D_i = \{1, 2, \dots, N\}$ (row position for queen i)

Constraints: $Q_i \neq Q_j$ (different rows)

$|Q_i - Q_j| \neq |i - j|$ (not on same diagonal)

4-Queen solution: [2, 4, 1, 3]

(Queen 1 in row 2, Queen 2 in row 4, Queen 3 in row 1, Queen 4 in row 3)

The key insight: since one queen is placed per column, the column constraint is automatically satisfied. We only need to check row and diagonal constraints.

4.3 Backtracking for CSPs

Backtracking is the fundamental algorithm for solving CSPs. It assigns values to variables one at a time, and when a partial assignment violates a constraint, it backtracks to the most recent variable and tries a different value.

Basic Backtracking Algorithm:

1. Select an unassigned variable X_i .
2. Try each value v in D_i :
 - a. If assigning $X_i = v$ is consistent with all constraints involving X_i :
 - i. Add $X_i = v$ to the assignment.
 - ii. Recursively attempt to assign remaining variables.
 - iii. If recursive call succeeds $\rightarrow$ return solution.
 - iv. If recursive call fails $\rightarrow$ remove $X_i = v$ (backtrack).
3. If all values tried and all fail $\rightarrow$ return failure (backtrack further).

```
N-Queens Backtracking (4x4):  
Col 1: try row 1  $\rightarrow$  place queen at (1,1)  
Col 2: try row 1  $\rightarrow$  conflict. try row 2  $\rightarrow$  conflict. try row 3  $\rightarrow$  OK  
Col 3: try row 1  $\rightarrow$  conflict. try row 2  $\rightarrow$  conflict. try row 3  $\rightarrow$  conflict.  
try row 4  $\rightarrow$  conflict.  
 $\rightarrow$  Backtrack to Col 2: try row 4  $\rightarrow$  OK  
Col 3: try row 1  $\rightarrow$  conflict. try row 2  $\rightarrow$  OK  
Col 4: try row 1  $\rightarrow$  conflict. try row 2  $\rightarrow$  conflict. try row 3  $\rightarrow$  conflict.  
try row 4  $\rightarrow$  conflict.  
 $\rightarrow$  Backtrack ... (continues until solution [2,4,1,3] found)
```

Improving Basic Backtracking:

- **Fail-First (MRV):** Select the variable with the Minimum Remaining Values (fewest legal values left). Detects failures earlier.
- **Degree Heuristic:** Among tied variables, choose the one involved in most constraints with unassigned variables.
- **Least Constraining Value:** When choosing a value, prefer the one that rules out fewest options for neighbours — keeps future choices open.

4.4 Constraint Propagation

Constraint propagation uses the constraints to reduce the domain of variables — eliminating values that cannot possibly be part of any solution. This reduces the search space before or during backtracking.

Arc Consistency (AC-3)

An arc (X_i, X_j) is arc-consistent if for every value in D_i , there exists at least one compatible value in D_j that satisfies the constraint between X_i and X_j . If not arc-consistent, values are removed from D_i .

Arc Consistency Example:

$X_i = \{1, 2, 3\}$, $X_j = \{1, 2\}$, Constraint: $X_i \neq X_j$

For $X_i=1$: X_j can be 2 ✓

For $X_i=2$: X_j can be 1 ✓

For $X_i=3$: X_j can be 1 or 2 ✓

All values in X_i are arc-consistent → no values removed.

Now suppose $X_j = \{2\}$ only:

For $X_i=2$: X_j must $\neq 2$ → no valid value in X_j → remove $X_i=2$

D_i reduced to $\{1, 3\}$

- **AC-3 Algorithm:** Maintains a queue of arcs to check. When a domain is reduced, re-add all arcs pointing to that variable.
- **Effect:** Can reduce domains significantly before search begins, or during backtracking — dramatically speeding up the search.

4.5 Look-Ahead Strategies

Look-ahead strategies check the impact of an assignment on future variables before committing to it — detecting failures earlier than basic backtracking.

Forward Checking

After assigning a value to X_i , forward checking propagates this constraint: for each unassigned variable X_j that is constrained with X_i , remove values from D_j that are now inconsistent. If any D_j becomes empty → backtrack immediately.

Forward Checking in N-Queens (4×4):

Assign $Q_1 = 1$ (queen in row 1, column 1)

Forward check: remove row 1 and diagonal values from D_2, D_3, D_4

$D_2 = \{3, 4\}$ (rows 1,2 eliminated by row/diagonal)

$D_3 = \{2, 4\}$

$D_4 = \{2, 3\}$

→ Already reduced the domains before exploring further!

Constraint Propagation During Search (MAC)

Maintaining Arc Consistency (MAC) runs AC-3 after every assignment during backtracking search — not just forward checking but full arc consistency propagation. More powerful than forward checking but more expensive per step.

Strategy	When Applied	Power	Cost
Basic Backtracking	Only checks current assignment	Low	Minimal

Strategy	When Applied	Power	Cost
Forward Checking	After each assignment	Medium	Low extra cost
MAC (Arc Consistency)	After each assignment, full propagation	High	Higher per step, fewer backtracks

5. Quick Revision — Unit IV at a Glance

5.1 All Search Algorithms Compared

Algorithm	Type	Complete?	Optimal?	Memory	Key Feature
Hill Climbing	Local	No	No	$O(b)$	Greedy local moves — fast but gets stuck
VND	Local	No	No	$O(b)$	Switches neighbourhoods to escape local optima
Beam Search	Heuristic	No	No	$O(W \times B)$	Keeps only top W nodes — memory bounded
Tabu Search	Metaheuristic	No	No	$O(\text{tabu list})$	Memory prevents revisiting — escapes local optima
A*	Optimal	Yes	Yes*	$O(b^d)$	$f=g+h$; optimal with admissible h ; uses OPEN/CLOSED
IDA*	Optimal	Yes	Yes*	$O(bd)$	Iterative depth limit on f — linear memory
RBFS	Optimal	Yes	Yes*	$O(bd)$	Recursive, remembers best f of abandoned branches

* Optimal when heuristic $h(n)$ is admissible.

5.2 Planning Approaches Summary

Approach	Space Searched	Ordered?	Handles Goals?	Interacting
FSSP (Forward)	World states (forward)	Total order	No	
BSSP (Backward)	World states (backward)	Total order	No	
Goal Stack Planning	Stack of goals/actions	Total order	Limited anomaly)	(Sussman)

Approach	Space Searched	Ordered?	Handles Goals?	Interacting
Plan Space (POP)	Partial plans	Partial order	Yes — Commitment	Least

5.3 CSP Techniques Summary

Technique	What it Does	When Applied
Backtracking	Assign values one by one; undo on conflict	Core search algorithm for CSPs
MRV Heuristic	Pick variable with fewest remaining values first	Variable ordering in backtracking
Forward Checking	Remove inconsistent values from future domains after each assignment	During backtracking, after each assignment
Arc Consistency (AC-3)	Ensure every value in X_i has compatible value in X_j for all constraints	Pre-processing or during search (MAC)
Look-ahead (MAC)	Full constraint propagation after every assignment	During backtracking — most powerful

5.4 Key One-Liners

- **Heuristic Function:** $h(n)$ = estimated cost from n to goal. Admissible: never over-estimates.
- **Hill Climbing:** Greedy local search — always moves uphill. Fails at local maxima, plateaus, ridges.
- **VND:** Escapes local optima by systematically switching to different neighbourhood structures.
- **Beam Search:** BFS but keeps only top W nodes per level. Memory-efficient, not optimal.
- **Tabu Search:** Local search with a tabu list memory — prevents cycling, escapes local optima.
- **A^* :** $f = g + h$. Optimal and complete with admissible h . Uses OPEN and CLOSED lists.
- **IDA*:** A^* with iterative deepening — linear memory, same optimality as A^* .
- **RBFS:** Recursive A^* — linear memory, saves best f of abandoned branches for later return.
- **AND-OR Trees:** OR node = any one child; AND node = all children must be solved.
- **STRIPS:** Planning language: State (facts) + Goal + Actions (Precond + Add + Delete).
- **Goal Stack Planning:** Stack-based planner — prone to Sussman anomaly with interacting goals.
- **Plan Space (POP):** Partial-order plans — minimum necessary ordering, handles interacting goals, supports parallelism.
- **N-Queens CSP:** Variables = queen columns, domains = rows, constraints = no same row/diagonal.
- **Backtracking:** Assign → check → backtrack on failure. Core CSP algorithm.
- **Forward Checking:** After assignment, remove inconsistent values from future domains immediately.

- **Arc Consistency:** For every value in X_i , at least one compatible value in X_j . AC-3 enforces this.

5.5 YouTube Links

- ▶ **Heuristic Search & Functions:** <https://www.youtube.com/watch?v=xaVTpt58bFI>
 - ▶ **Hill Climbing Algorithm:** <https://www.youtube.com/watch?v=z2y1sMrHKDw>
 - ▶ **A* Algorithm:** <https://www.youtube.com/watch?v=95s0b0HBe3w>
 - ▶ **Problem Decomposition & AND-OR Trees:** <https://www.youtube.com/watch?v=JL9g1BBSunA>
 - ▶ **Linear Planning:** <https://www.youtube.com/watch?v=WIG0H0u8aCg>
 - ▶ **Non-Linear Planning:** https://www.youtube.com/watch?v=kHwjIOZe_jY
-